

Python Script

30.09.2026, 14:16:53

```
1      """
2      Лабораторная работа № 1.
3      Задание 3: вычисление  $\ln c$  с нормализацией аргумента ( $c = m \cdot 2^k$ )
4      и сравнение числа итераций с ненормализованным вариантом.
5      """
6
7      import math
8
9      EPS = 1e-6
10     N_MAX = 100_000
11
12
13     def ln_series(c, eps=EPS, n_max=N_MAX):
14         """
15         Натуральный логарифм через ряд
16          $\ln c = 2 \cdot (y + y^3/3 + y^5/5 + \dots)$ ,  $y = (c - 1)/(c + 1)$ 
17         Без нормализации аргумента.
18         Возвращает (значение, число итераций).
19         """
20         if c <= 0:
21             raise ValueError("Аргумент логарифма должен быть положительным")
22
23         y = (c - 1) / (c + 1)
24         y2 = y * y
25         power = y      # текущая степень  $y^{(2k+1)}$ 
26         total = 0.0
27
28         for k in range(n_max):
29             term = 2 * power / (2 * k + 1)
30             if abs(term) < eps:
31                 return total, k
32             total += term
33             power *= y2
34
35         raise RuntimeError("Точность не достигнута за n_max итераций")
36
37
38     def _ln_series_normalized_argument(c, eps=EPS, n_max=N_MAX):
39         """
40         Вспомогательная функция:  $\ln c$  для  $c$  в диапазоне  $[0.5, 1)$ 
41         ( $|y| \leq 1/3$ , ряд сходится быстро).
42         """
43         y = (c - 1) / (c + 1)
44         y2 = y * y
45         power = y
46         total = 0.0
47
48         for k in range(n_max):
49             term = 2 * power / (2 * k + 1)
50             if abs(term) < eps:
51                 return total, k
52             total += term
53             power *= y2
54
55         raise RuntimeError("Точность не достигнута за n_max итераций")
56
57
58     def ln_normalized(c, eps=EPS, n_max=N_MAX):
59         """
60         Натуральный логарифм с нормализацией аргумента:
61          $c = m \cdot 2^k$ ,  $m$  in  $[0.5; 1)$ 
62          $\ln c = \ln m + k \cdot \ln 2$ 
63         Возвращает (значение, суммарное число итераций).
64         """
65         if c <= 0:
66             raise ValueError("Аргумент логарифма должен быть положительным")
67
68         # Приводим  $c$  к виду  $m \cdot 2^k$ ,  $m$  in  $[0.5; 1)$ 
69         k = 0
70         m = c
71         while m >= 1.0:
72             m *= 0.5
73             k += 1
74         while m < 0.5:
75             m *= 2.0
```

```

76         k -= 1
77
78     ln_m, it_m = _ln_series_normalized_argument(m, eps)
79     ln_2, it_2 = _ln_series_normalized_argument(2.0, eps)
80
81     return ln_m + k * ln_2, it_m + it_2
82
83
84 if __name__ == "__main__":
85     values = [0.1, 0.5, 1.5, 5, 10, 50, 100, 1000]
86
87     print(f"{'c':>10}{'ln(c)':>20}{'эталон':>20}"
88           f"{'итер. без норм.':>18}{'итер. с норм.':>18}")
89     print("-" * 86)
90
91     for c in values:
92         val_plain, it_plain = ln_series(c, EPS)
93         val_norm, it_norm = ln_normalized(c, EPS)
94         ref = math.log(c)
95
96         print(f"{'c':>10}{'val_plain':>20.10f}{'ref':>20.10f}"
97               f"{'it_plain':>18}{'it_norm':>18}")
98
99     print("\nПроверка точности:")
100    for c in values:
101        val_norm, _ = ln_normalized(c, EPS)
102        err = abs(val_norm - math.log(c))
103        print(f" c = {c:>8}: |ln(c) - math.log(c)| = {err:.3e}")

```